

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment 3

August 2026

***Instructions:** This is a group assignment. Each group must implement the Verilog designs, write suitable testbenches, and demonstrate the working designs and their behavioral simulations to a Teaching Assistant for evaluation. The designs must be implemented using combinational logic and hierarchical module interconnection.*

Question 1: 3-bit Array Multiplier

Design and simulate a combinational circuit that multiplies two unsigned 3-bit binary numbers using partial-product generation and Half/Full Adders.

1. Module Specification

Create a module named

`array_multiplier_3bit`

with the following interface:

Signal	Width	Direction
A	3 bits	Input
B	3 bits	Input
P	6 bits	Output

The circuit must compute

$$P = A \times B.$$

The Verilog multiplication operator `*` must **not** be used in the design module.

2. Partial-Product Generation

Let

$$A = A_2A_1A_0, \quad B = B_2B_1B_0.$$

Generate the nine partial products

$$p_{ij} = A_i \cdot B_j$$

using AND logic.

The partial-product arrangement is:

$$\begin{array}{rrrr} & & A_2B_0 & A_1B_0 & A_0B_0 \\ & A_2B_1 & A_1B_1 & A_0B_1 & \\ A_2B_2 & A_1B_2 & A_0B_2 & & \end{array}$$

The partial products must then be added using Half Adder and Full Adder modules.

Students may reuse the `half_adder` and `full_adder` modules developed in Verilog Assignment 1.

3. Required Adder Connections

The partial products must be combined as follows.

1. The least significant product bit is

$$P_0 = p_{00}.$$

2. Add p_{10} and p_{01} using a Half Adder:

$$(P_1, C_1) = HA(p_{10}, p_{01}).$$

3. Add p_{20} , p_{11} , and p_{02} using a Full Adder:

$$(S_2, C_{2a}) = FA(p_{20}, p_{11}, p_{02}).$$

Then combine S_2 and C_1 using a Half Adder:

$$(P_2, C_{2b}) = HA(S_2, C_1).$$

4. Add p_{21} , p_{12} , and C_{2a} using a Full Adder:

$$(S_3, C_{3a}) = FA(p_{21}, p_{12}, C_{2a}).$$

Then combine S_3 and C_{2b} using a Half Adder:

$$(P_3, C_{3b}) = HA(S_3, C_{2b}).$$

5. Finally, add p_{22} , C_{3a} , and C_{3b} using a Full Adder:

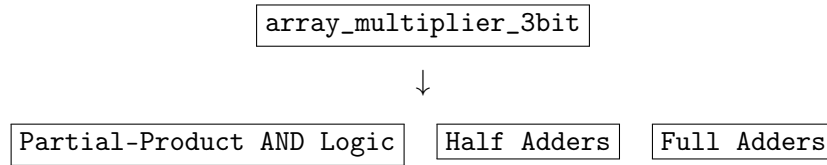
$$(P_4, P_5) = FA(p_{22}, C_{3a}, C_{3b}).$$

Thus, the complete output is

$$P = P_5P_4P_3P_2P_1P_0.$$

4. Design Hierarchy

The required design hierarchy is:



The multiplier must therefore be constructed by explicitly generating the partial products and connecting the required adder modules.

5. Examples

Some example input-output combinations are:

A	B	P
000	101	000000
001	111	000111
010	011	000110
101	011	001111
111	110	101010
111	111	110001

For example,

$$A = 101 = 5, \quad B = 011 = 3$$

must produce

$$P = 001111 = 15.$$

6. Testbench Requirement

Write a testbench for the `array_multiplier_3bit` module.

The testbench must test multiple combinations of A and B, including:

- multiplication by zero;
- multiplication by one;
- equal operands;
- different operands; and
- the maximum input values.

Students are encouraged to verify all $8 \times 8 = 64$ possible input combinations. The behavioral simulation waveform must show at least:

A, B, P.

Question 2: Carry-Save Adder for Three 4-bit Numbers

Design and simulate a combinational circuit that adds three unsigned 4-bit binary numbers using a **Carry-Save Adder (CSA)** followed by a final Ripple-Carry Adder.

1. Top-Level Specification

Create a module named

`carry_save_adder_3x4`

with the following interface:

Signal	Width	Direction
A	4 bits	Input
B	4 bits	Input
C	4 bits	Input
Result	6 bits	Output

The circuit must compute

$$\text{Result} = A + B + C.$$

The three operands must **not** be added using a single Verilog expression such as

`assign Result = A + B + C;`

in the design module.

Instead, the circuit must use the Carry-Save structure described below.

2. Carry-Save Stage

Create a module named

`carry_save_stage_4bit`

with three 4-bit inputs

$A[3:0], \quad B[3:0], \quad C[3:0],$

and outputs

$S[3:0], \quad \text{Carry}[3:0].$

The module must instantiate **four Full Adders**, one for each bit position. For each bit i ,

$$FA_i(A_i, B_i, C_i)$$

must generate

$$S_i$$

and

$$\text{Carry}_i.$$

The four Full Adders operate independently:

Full Adder	Input 1	Input 2	Input 3	Outputs
FA0	A[0]	B[0]	C[0]	S[0], Carry[0]
FA1	A[1]	B[1]	C[1]	S[1], Carry[1]
FA2	A[2]	B[2]	C[2]	S[2], Carry[2]
FA3	A[3]	B[3]	C[3]	S[3], Carry[3]

Important: the carry output of one Full Adder must **not** be connected to the next Full Adder within the Carry-Save stage.

Thus, unlike a Ripple-Carry Adder, there is no carry propagation between the four Full Adders in this stage.

3. Formation of the Two Intermediate Operands

The Carry-Save stage produces two intermediate values.

The Sum vector is extended to 5 bits as

$$S_{\text{ext}} = \{1'b0, S[3:0]\}.$$

The Carry vector must be shifted left by one bit because every carry generated at bit position i has a weight of 2^{i+1} :

$$Carry_{\text{shifted}} = \{Carry[3:0], 1'b0\}.$$

Therefore,

$$A + B + C = S_{\text{ext}} + Carry_{\text{shifted}}.$$

4. Final Ripple-Carry Adder

Create a module named

`ripple_carry_adder_5bit`

which adds the two 5-bit intermediate values:

$$S_{\text{ext}}$$

and

$$Carry_{\text{shifted}}.$$

The Ripple-Carry Adder must be constructed using **five instances of the full_adder module**.

Its final output is the 6-bit value

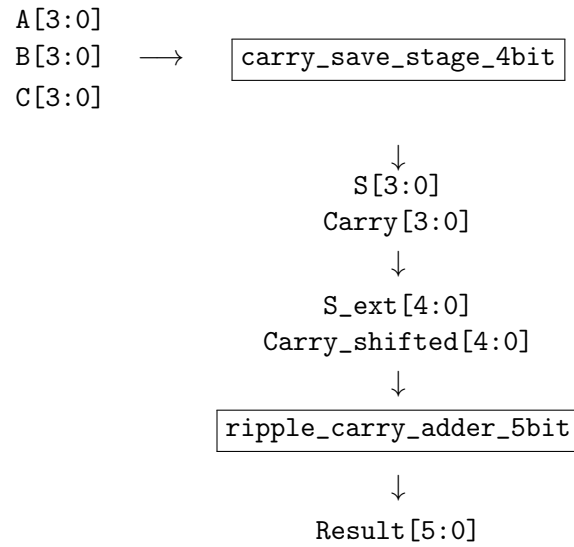
$$\text{Result}[5:0].$$

The carry connection must follow:

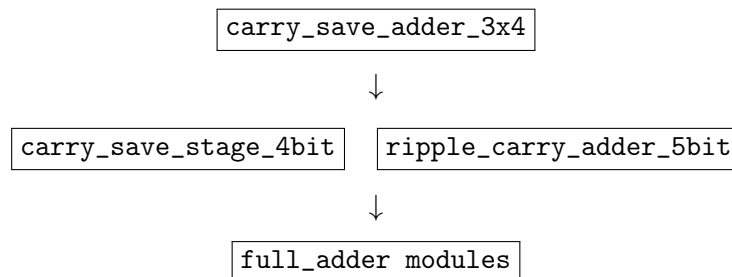
$$C_0 = 0 \longrightarrow \text{FA0} \xrightarrow{C_1} \text{FA1} \xrightarrow{C_2} \text{FA2} \xrightarrow{C_3} \text{FA3} \xrightarrow{C_4} \text{FA4} \xrightarrow{C_5} \text{Result}[5].$$

5. Complete Design Hierarchy

The complete Carry-Save Adder structure is:



The module hierarchy is therefore:



6. Examples

Some example input-output combinations are:

A	B	C	Result
0000	0000	0000	000000
0001	0010	0011	000110
0011	0101	0110	001110
1001	0011	0100	010000
1111	1111	0000	011110
1111	1111	1111	101101

For example,

$$A = 0011 = 3, \quad B = 0101 = 5, \quad C = 0110 = 6$$

must produce

$$\text{Result} = 001110 = 14.$$

7. Testbench Requirement

Write a testbench for the top-level `carry_save_adder_3x4` module.

The testbench must include:

- all-zero inputs;
- small input values;
- different values for all three operands;
- cases that produce carries across several bit positions; and
- the maximum input combination $A = B = C = 1111$.

The behavioral simulation waveform must show at least:

A, B, C, Result.

Students are encouraged to also display the internal **S** and **Carry** vectors during demonstration.

Verilog Design Requirement

Both questions must follow the specified hierarchical module organization.

Students may use gate primitives or continuous Boolean assignments inside the Half Adder and Full Adder modules. However:

- the Array Multiplier must generate and combine the individual partial products explicitly;
- the Verilog multiplication operator `*` must not be used to implement the multiplier;
- the Carry-Save Adder must use Full Adder instances for the carry-save reduction stage;
- the final addition in the Carry-Save Adder must use the specified Ripple-Carry Adder; and
- the complete arithmetic function must not be replaced by a single behavioral arithmetic expression.

All designs must be purely combinational. No clocked or sequential logic is required.

Demonstration is mandatory. Each group must demonstrate both working designs and their behavioral simulations to a Teaching Assistant.

Students may be asked to explain:

- partial-product generation in the Array Multiplier;
- how the Half Adders and Full Adders combine the partial products;
- why carries are not propagated between Full Adders in the Carry-Save stage;
- why the Carry vector must be shifted left by one position; and
- how the final Ripple-Carry Adder produces the final result.

Submission

Submit all Verilog design source files and testbenches in a single zipped folder named

`VerilogAssignment3_{Group_Number}.zip`.

The submission must contain the required modules for:

- `half_adder`;
- `full_adder`;
- `array_multiplier_3bit`;
- testbench for `array_multiplier_3bit`;
- `carry_save_stage_4bit`;
- `ripple_carry_adder_5bit`;
- `carry_save_adder_3x4`; and
- testbench for `carry_save_adder_3x4`.

Each source file must contain the group number, names and roll numbers of all group members, assignment number, and date of submission.